Linux System Hardening: Checklist and Guided Projects

A practical hardening checklist plus four guided projects with real deliverables, from a single baseline-hardened server to a monitored multi-host stack.

Hardening checklist

Accounts and access

- Disable root SSH login and password authentication (`PermitRootLogin no`, `PasswordAuthentication no`)
- Use SSH keys (ed25519) and restrict users with `AllowUsers`
- Enforce a password policy via `pam_pwquality` and `faillock`
- Configure `sudo` with least privilege; audit users with UID 0

Network

- Default-deny firewall (`nftables`, `ufw` or `firewalld`); open only required ports
- Disable unused services (`systemctl list-unit-files --state=enabled`)
- Harden kernel network settings via sysctl (rp_filter, disable ICMP redirects, enable SYN cookies)
- Install `fail2ban` or `sshguard`

System and kernel

- Apply automatic security updates
- Separate and mount-restrict partitions (`/tmp`, `/var`, `/home` with `nodev,nosuid,noexec` where possible)
- Enable SELinux or AppArmor in enforcing mode
- Restrict core dumps, `dmesg` and `kptr_restrict`
- Enable Secure Boot and a GRUB password if physical access matters

Files and permissions

- Find SUID/SGID binaries and world-writable files
- Set a restrictive `umask` (027)
- Protect `/etc/shadow`, SSH keys and cron directories

Logging and monitoring

- Enable `auditd` with rules for auth, sudo and file changes
- Ship logs to a remote host
- Install a file integrity checker (AIDE)
- Run `rkhunter` or `chkrootkit` and Lynis regularly

Verification

- Run Lynis and OpenSCAP against the CIS Benchmark for your distro
- Re-scan after changes and record your score improvement

Guided projects

Project 1: Secure a fresh server (beginner)

Goal: Take a default Ubuntu, Debian or Rocky VM from insecure to baseline-hardened.

Steps: Baseline Lynis scan, harden SSH, configure the firewall, create a non-root admin user, enable unattended upgrades, install fail2ban, then re-scan.

Deliverables:

- Before/after Lynis reports with score comparison
- A `harden.sh` script that reproduces your changes
- A one-page changelog explaining each change and why

Project 2: CIS Benchmark compliance audit (intermediate)

Goal: Measure and remediate a system against CIS Level 1.

Steps: Run OpenSCAP or `cis-cat`, triage failures by risk, remediate manually (to learn), then automate with Ansible and re-scan.

Deliverables:

- HTML compliance report (before and after)
- Ansible playbook or role that applies the fixes idempotently
- Exceptions register: controls you skipped and the justification

Project 3: Mandatory access control and auditing (intermediate)

Goal: Confine a service and detect misuse.

Steps: Put SELinux or AppArmor in enforcing mode, confine a web server (nginx or Apache), write or tune a policy for a custom app, configure `auditd` rules, then simulate an attack (for example, writing outside the web root) and trace the logs.

Deliverables:

- Custom policy or profile file
- `audit.rules` file with explanations
- Incident write-up showing the denied action and how you detected it

Project 4: Hardened, monitored production stack (advanced)

Goal: Build a multi-host environment with continuous verification.

Steps: Provision 2-3 VMs (web, DB, log server), harden them with Ansible, segment the network with nftables, set up central logging (rsyslog, Loki or ELK), run AIDE integrity checks on a cron job or systemd timer, add alerting on failed logins and file changes, then test yourself with Nmap and Lynis.

Deliverables:

- Architecture diagram
- Git repo with infrastructure code and README
- Monitoring dashboard screenshots and alert rules
- Final security assessment report with findings and remediation

Capstone (optional)

Document everything as a **portfolio-ready runbook**: threat model, hardening standard, automation repo and audit evidence.

Tools you'll use

Lynis, OpenSCAP, Ansible, auditd, AIDE, fail2ban, nftables, SELinux/AppArmor, Nmap